

CoQuí scGW on GPU: Performance Diagnosis and Optimization Plan

Why the port runs at CPU speed — and the path to 5–8×

Miguel Morales (with Claude)

Flatiron Institute — rusty cluster — branch `gpu`, `ad9bc48`→`765754c`

July 26, 2026

Abstract

We benchmarked CoQuí’s THC-based self-consistent GW on rusty CPU (Genoa) and GPU (A100, H100) nodes using Si supercell problems up to $\Pi \approx 1.4$ TB ($4 \times 4 \times 4$ k -grid, 500 bands). The GPU compute kernels are excellent — the screened-interaction Dyson solve runs 16–23× faster than the CPU code and sits at its roofline — but end-to-end the port shows essentially no speedup (1.0–1.5× at equal node count). The cause is quantified precisely: $\sim 70\%$ of GPU wall time is data movement — host-staged all-to-all redistributions in the $\tau \leftrightarrow \omega$ transforms, a host-resident gemm in the THC basis transform, per-iteration re-gathers of iteration-invariant data, raw `cudaMalloc` churn, and host-only SCF phases. A single-node (network-free) GPU run proves the cost is PCIe staging, not the interconnect. No algorithmic change is required: the existing transpose-based transform is communication-optimal. We propose a prioritized engineering plan (device-direct chunked redistribution, device memory pool, device-resident basis transforms, invariant caching, device Dyson stage) with a projected $\approx 5\times$ node-for-node speedup at the $2 \times 2 \times 2 / 500$ -band scale, growing to 5–8× at production scale. The first and largest item is now implemented and measured: a device-direct pairwise exchange cuts `update_w` from 163 to 99 s/iter at 8 A100 and from 92 to 62 s/iter at 16, with energies unchanged digit for digit. During the study we also found and fixed a correctness regression that had made the THC-HF Fock matrix processor-count-dependent on *both* CPU and GPU since 2026-05-12; all configurations now agree to 10^{-13} .

1 Methodology

All runs use one source tree (branch `gpu`, HEAD `765754c`) and three binaries: CPU (icelake SLATE), CUDA sm80 (A100), CUDA sm90 (H100/H200). Identical physics inputs throughout: $\beta = 2000$, $\Lambda = 1200$, IR grid “high” ($n_\tau = n_\omega = 137$), THC `ecut`=150, `thresh`= 10^{-5} , damping mixing 0.7; three SCF iterations (two at kp444). Phase timers are device-synchronized. Because THC interpolating-point selection depends on the processor grid (finding R1), all cross-configuration comparisons load a single pre-built ERI file (`save`= mechanism). Every timed configuration was validated against a serial CPU reference energy.

Node type	Configuration	FP64 DGEMM/node	Host RAM
CPU (ccq, Genoa)	$2 \times 48c$ AMD Genoa, 96 ranks	≈ 4.5 TF	1.5 TB
GPU A100	$4 \times$ A100-SXM4-80GB (NVLink)	39 TF vec / ≈ 78 TF TC	1 TB
GPU H100	$8 \times$ H100-PCIe-80GB	≈ 270 TF TC	1 TB

Table 1: Hardware. Fair yardstick: gemm-dominated phases should run ≈ 10 – $17\times$ faster on an A100 node than a Genoa node (≈ 25 – $60\times$ on $8\times$ H100), minus communication.

2 Correctness findings

Timing data is only meaningful on correct code; the study began by validating physics and found the following.

#	Status	Defect	Impact
B1	fixed (765754c)	<code>_aux_to_primary_impl</code> X-factor cache (introduced 734ae79, May 12) reused the <i>P</i> -sliced cache as the right gemm factor when <i>ip=iq</i> ; the right factor must be sliced with <code>Q_rng</code> .	THC-HF Fock processor-count dependent on CPU <i>and</i> GPU since May 12 ($\ \Delta F\ _\infty$ up to 0.3 a.u. between rank counts; GPU total energies off $\times 1000$). GW Σ provably unaffected (its grid keeps $np_P=np_Q=1$; bit-identical across ranks). All May benchmark energies were wrong. Post-fix: CPU 1/2/4/8 ranks and GPU A100(16r)/H100(1–16r) agree to 10^{-13} .
B2	fixed (80f4a5d)	<code>TimerManager</code> aborts in <code>update_w<HOST></code> : the timer dump queries DEVICE-only timer names.	Every freshly built CPU binary crashed in iteration 1.
B3	open / workaround in place	Device ISDF/THC builder seg-faults at ≥ 16 ranks: UCX “invalid permissions for mapped object” in the <code>Zqur</code> stage (device buffer reaches a host-only MPI path; forcing UCX CUDA transports does not help). Pre-existing (May 16-rank run died identically).	Blocks all large GPU runs. Workaround: pre-build the ERI at ≤ 8 GPU ranks or on CPU, <code>save=</code> , load everywhere (also guarantees identical integrals across runs).
B6	open / workaround in place	Device ISDF interpolating-point selection (<code>chol_metric_impl<DEVICE_MEMORY></code> , <code>thc.icc:1073</code>) selects <i>zero</i> Cholesky vectors on sm90/H100 and aborts; identical source and input on sm80/A100 build normally.	Blocks THC/ERI construction on H100. Attribution is confirmed: A100 yields $N_p=1275$ and $E=0.514582271575844$ <i>bit-identically</i> before and after the allocator/pool commits, and the CPU path agrees — the failure tracks architecture, not code version. Unseen until now because every H100 run loaded a prebuilt ERI. Workaround: build the ERI on A100 or CPU (<code>save=</code>) and load it on H100. Suspects: <code>cuSOLVER</code> / <code>SLATE</code> (GPU_H100 build) / <code>cuTENSOR</code> behaviour on sm90.
B4	open	μ -search bracketing walk (<code>scf_common.cpp:190–202</code>) is silent and uncapped (0.2 a.u. steps, no iteration limit, no logging).	Turned B1’s wrong Σ into a 4 h wall-limit hang. Needs a cap + diagnostics.
B5	open	<code>compute_eigenspectra</code> allocates the full $\Sigma(\omega)$ ($n_\omega n_s n_k n_b^2$) <i>per rank</i> : 35 GB/rank at kp444/500b.	kp444 CPU jobs OOM at full rank density (forced to 24 ranks/node). Distribute or chunk over ω .
R1	reproducibility	THC interpolating-point selection depends on the processor grid ($N_p = 4482/4483/4471$ across rank counts).	$\sim 10^{-4}$ a.u. total-energy spread between rank counts at thresh 10^{-5} . Comparisons must share a saved ERI.

Table 2: Defects found during the study.

3 Measured performance (corrected code)

3.1 Si 2×2×2, 500 bands ($N_p \approx 4483$, $\Pi \approx 177$ GB)

Seconds per SCF iteration	CPU		GPU		
	1n·96r	2n·192r	1n·8×H100	2n·8×A100	4n·16×A100
update_w ($\Pi \rightarrow W$)	343	235	185	162	91
eval Π (RPA)	32	21	21	20	11
$\tau \leftrightarrow \omega$ FTs (redistributes)	37	24	133	111	60
W-Dyson solve (LU + gemms)	273	191	11.6	17	12.8
W D→H mirror + allocs	1	2	17.5	13.4	6.8
THC-GW Σ	88	55	45	50	33
$G_{ij} \rightarrow G_{uv}$ (host-bound gemm)	27	19	17.5	18	9.7
Wall, 3 iters + setup	1423	1018	941	1009	632
host-only phases (Dyson/ μ , mix, write)	109	132	215	337	240

Table 3: kp222/500b phase breakdown. Single node, *no network*: 8×H100 vs. 96-core Genoa is 1.5× wall while its W-solve alone is 23× faster — the 133 s of “communication” is pure PCIe staging + intra-node host MPI. The staging, not the wire, is the problem.

3.2 Where a GPU iteration goes (2n·8×A100, 215 s solver time)

Component	s/iter	class
$\tau \leftrightarrow \omega$ redistributes (host-staged)	111	transfer waste
Necessary device compute (W solve, Π , Σ gemms, FTs)	≈50	device compute
$G_{ij} \rightarrow G_{uv}$ host gemm + per- (t, s, k) H2D	18	host-bound compute
W D→H mirror + 23 GB device allocations	13	transfer/alloc waste
Σ host reduce + shm accumulate	≈12	host-bound
Z re-gather (12.5 s at 16r; 187 s/iter at kp444)	4	transfer waste

Table 4: Waste decomposition. In addition, each run carries ≈200–300 s of host-only SCF machinery (Dyson G -solve, μ search, eigenspectra, mixing, rank-0 serial HDF5 of a 35 GB checkpoint) that a 192-rank CPU job hides but an 8-rank GPU job cannot.

3.3 Anatomy of one redistribute

Each `tau_to_w/w_to_tau` performs two full redistributions of the Π/W tensor (t -distributed $\rightarrow PQ$ -distributed $\rightarrow \omega$ -distributed); four per iteration. For device arrays, `redistribute_alltoallv` (`nda_utils.hpp:715`) does, per peer: strided pack into a fresh device temporary (`raw cudaMalloc`) $\rightarrow$ bulk D2H into a host vector $\rightarrow$ host `MPI_Alltoallv` $\rightarrow$ H2D $\rightarrow$ device read-modify-write unpack. That is four PCIe crossings of ~22 GB/rank per transform pair, $O(n_{\text{ranks}})$ device allocations each, and ~4 s per 23 GB buffer allocation. The May CUDA-aware attempt (0e8fc80) was reverted for the right symptom and the wrong reason: it kept *unchunked* full-size device staging buffers, doubling device memory $\rightarrow$ OOM. The pattern is sound; it needs chunking and a pool.

3.4 Production scale: Si 4×4×4, 500 bands ($\Pi \approx 1.42$ TB)

CPU reference (8 nodes, 192 ranks, 24/node $\times$ 4 threads): **update_w** 1427 s/iter (W-solve 944, **Z re-gather** 187, FTs 134), **THC-GW** 383 s/iter ($G_{ij} \rightarrow G_{uv}$ 118, $\bar{\Sigma}_{uv} \rightarrow \Sigma_{ij}$ 146); wall 4213 s for 2

iterations. Roofline: the W solve is 12.7 Pflop/iter $\rightarrow \approx 82$ s on 64 A100. Every overhead term scales *worse* than compute at kp444 (redistribute volume $\times 8$; Z-gather $\times 8$ with a full- N_p^2 allreduce per q -point). GPU runs at this scale (64 $\times$ A100, 48 $\times$ H200, on pre-built ERIs) are queued as before/after validation; they do not change the plan.

4 Results of the first fixes (measured)

4.1 P1 and the leakage diagnostic

Two changes account for a 2.2 $\times$ reduction of `update_w` at 8 A100 ranks. P1 (below) removed the host staging from the redistribute. Instrumenting what remained then showed that the largest term in the $\tau \leftrightarrow \omega$ phases was not communication at all but `IAFT::check_leakage`, a *diagnostic*: on device input it began with `nda::to_host` of the whole tensor (18.5 GB/rank) and then ran two gemms over it on one host core, four times per iteration — 28 s/iter. Its output is three coefficient rows, $3/n_\tau$ of the tensor, so it now contracts on the device and returns only those rows (817df17): 28.0 $\rightarrow$ 2.3 s/iter, diagnostic value unchanged. Note the same `to_host`-the-tensor pattern remains in `simple_dyson`, `scf_common`, `thc_sosex` and the embedding code.

Per SCF iteration, 2 nodes $\cdot$ 8 $\times$ A100	host staging	+P1	+device leakage
update_w	163.1	99.6	74.3
$\tau \rightarrow \omega$	52.0	21.0	10.8
$\omega \rightarrow \tau$	57.2	25.5	10.7
of which redistribute (4 calls)	—	18.7	18.2
of which <code>check_leakage</code>	—	28.0	2.3
SCF loop, 2 iterations	682.8	548.5	493.5

Table 5: Cumulative effect of the two fixes. Energies are unchanged throughout (0.48305038078408613, then 0.5300765029178911).

Within the exchange itself, pack and unpack cost 0.02 s each per call and the rest is time on the wire at 7.5–9.7 GB/s; a depth-2 software pipeline over the chunks changed nothing (100.1 vs 99.9 s/iter), confirming the exchange is bandwidth-bound rather than stalled. NCCL remains worth testing here, on the theory that it drives the same hardware better than UCX does for this pattern.

4.2 P1 in detail

Seconds per SCF iteration	2 nodes · 8×A100		4 nodes · 16×A100	
	host staging	device-direct	host staging	device-direct
update_w	163.1	99.6	92.2	61.6
$\tau \rightarrow \omega$	52.0	21.0	27.1	13.6
$\omega \rightarrow \tau$	57.2	25.5	32.2	17.5
W-Dyson solve	17.6	17.4	—	—
eval Π (RPA)	22.0	21.4	—	—
SCF loop, 2 iterations	682.8	548.5	429.2	370.5

Table 6: P1 measured: Si $2 \times 2 \times 2/500$ bands, shared ERI, two SCF iterations, identical binary with only the transport switched. Energies agree digit for digit (0.48305038078408613, then 0.5300765029178911 at 8 ranks). The phases that should not move do not. What remains in the transform phase is the IAFI gemms plus an exchange that still serializes pack/send/unpack per chunk.

5 Improvement plan

Ordered by impact $\div$ risk. Gains quoted against the 2n·8×A100 kp222/500b iteration (215 s solver + ~ 110 s/iter-equivalent host phases); P1/P4/P6 gains are larger at kp444.

P1 — Device-direct chunked redistribute. fixed (9860888, c2f8e71) measured –64 s/iter at 8 ranks, –31 s/iter at 16.

Implemented and now the default wherever the MPI advertises device-pointer support (MPIX_Query_cuda_support; COQUI_REDIST_DEVICE forces either path). The DEVICE path walks peers in a fixed pairwise schedule — round r sends to $\text{rank}+r$ and receives from $\text{rank}-r$ — and hands device pointers straight to CUDA-aware MPI. Only one peer block is in flight, so the extra device memory is bounded by a chunk budget (COQUI_REDIST_CHUNK_MB, default 1 GB per buffer) rather than by the tensor size: exactly the bound the May attempt lacked. Over-budget blocks split along their slowest dimension, and both sides stage through a C-ordered view of the same global index box, so they derive the same split with no extra metadata exchange. Measured (kp222/500b, shared ERI, A100): see Table 6. Bit-identical to host staging in every controlled comparison but one, which differed by 3×10^{-15} (round-off; total energies matched to all 16 digits). **NCCL was not needed** — and is not available in these builds (NCCL_LIBRARIES-NOTFOUND); plain CUDA-aware MPI over UCX suffices, including at 16 ranks, where the ISDF builder’s device-MPI crash (B3) notably does *not* appear. Remaining headroom: the exchange still serializes pack/send/unpack per chunk with a device synchronize between them, so the phase sits at 47 s (8 ranks) rather than the 12 s a bandwidth estimate allows; double-buffering across two streams is the next step.

P2 — Activate nda’s existing pooled allocator. built, opt-in, not usable at this scale (ae0486f, 975c912, 9860888).

Built as described below and wired into the SCF loop as a scoped reservation (utils::device_pool_guard, sized from COQUI_DEVICE_POOL_GB), with the ISDF builder’s free-memory query made pool-aware. It serves the redistribute staging buffers correctly (16 requests, zero fall-backs, 10^{-13} -identical energies), but reserving 3 GB inside the SCF loop at kp222/500b turned iteration 2 into a cudaErrorMemoryAllocation while allocating an 18.5 GB tensor with 36.5 GB reported free — the failure mode the design notes below warn about. With 18.5 GB

tensors on an 80 GB card the headroom is simply not there, and P1’s gain does not depend on it, so the pool stays off by default until the peak footprint shrinks. The analysis below stands and is worth keeping for that point. Every device scratch array is currently a raw `cudaMalloc/cudaFree` (`nda::cuarray = heap<Device> = mallocator`); a 23 GB allocation costs ~ 4 s and each redistribute makes $O(n_{\text{ranks}})$ of them. CoQuí already exposes the remedy — `memory::buffered_array<MEM, ...>`, backed by `static_fallback<dynamic_bucket<Device>` (`configuration.hpp:169-211`) — a first-fit pool with free-list coalescing, a process-wide static instance, and graceful fallback to `mallocator` on a pool miss. *No new allocator is needed.* **Why it isn’t already helping:** the pool is inert — `dynamic_bucket`’s default capacity is **8000 bytes** and nothing in the tree ever calls `resize()`, so even the four existing `buffered_array` uses (cuSPARSE glue) fall straight through to `cudaMalloc`. Work: (i) size the pool once at startup via `get_primary()->resize()` from a config knob (fraction of free VRAM); (ii) route the hot paths — either flip `device_array` (`configuration.hpp:105`) to the buffered handle `detail::buffered_handle_t<DEVICE_MEMORY>`, which pools every `memory::array<DEVICE_MEMORY, ...>` with no call-site type churn, or opt in per hot path to keep the giant per-iteration darrays on the plain heap. **Constraints:** `resize()` throws if any block is live (size before the first device allocation, never mid-run); first-fit with coalescing has no defragmentation, so mixing 20 GB darrays with thousands of small scratch blocks can fragment the pool and silently revert to `cudaMalloc` — log capacity and `maximum_memory()` to catch it; the static pool is neither thread-safe nor stream-ordered, which is fine today (one rank per GPU, single stream) but would require per-stream pools or `cudaMallocAsync` if P1’s chunking later overlaps pack/send/unpack across streams (the one case where `cudaMallocAsync` retains an edge). Note also that reserving a large pool shrinks the `cudaMemGetInfo` free memory the ISDF builder reads to choose its blocking (`thc_aux.icc:1189`): size the pool after ERI construction, or make that query pool-aware.

P3 — Device-resident basis transforms. $-25-30$ s/iter.

(Effort M, risk L.) In `_primary_to_aux_impl` the first gemm ($X \cdot G$) runs on the host (one core per rank) with a small H2D per (t, s, k) — which is why $G_{ij} \rightarrow G_{uv}$ costs the same 18 s on 8 GPUs as on 192 CPU cores. Stage G (4.4 GB at kp222; chunked over k/t at kp444) and the X collocation blocks to device once per iteration and run both gemms there. Same change serves Π and Σ ; drop the per- i host accumulate on the output side.

P4 — Cache iteration-invariant THC data. -5 s/iter (kp222); -180 s/iter (kp444).

(Effort S, risk L.) `thc_reader_t::Z()` re-gathers each q block every iteration with two scalar all-reduces, a barrier-bearing gather, a full- N_p^2 allreduce (only the local block is kept), and a fresh H2D. Z never changes during SCF: cache the device slice per local q (LRU if tight). Also cache the small IAFIT transform matrices (rebuilt + re-uploaded per call) and the $k \leftrightarrow R$ coefficients (recomputed up to $8\times$ /iter, each with a global barrier + internode allreduce).

P5 — Lazy W host mirror. -13 s/iter; -180 GB host RAM/node.

(Effort S, risk L.) `update_w` keeps `dW_qtPQ_dev` for the GPU Σ path yet still materializes the full host mirror each iteration for consumers absent in a pure scGW run. Make it lazy; batch the $(t, q) \rightarrow (q, t)$ transpose into one kernel.

P6 — Device Dyson stage; eigenspectra memory; μ cap. $-60-80$ s/iter-equiv.

(Effort M/L, risk M.) `simple_dyson` has no MEM template: $n_\omega n_k$ inversions of n_b^2 matrices, the μ search, and eigenspectra run on host across 8–64 ranks. Add a DEVICE path for the Dyson loop (batched `getrf/getrs` over ω), distribute/chunk the per-rank $\Sigma(\omega)$ buffer (B5), cap and

log the μ walk (B4). The host `geev` is cheap once the redundant allocation is gone.

P7 — LU-solve instead of explicit inverse in the W solve. −6–7 s/iter.

(Effort S, risk L.) $W = [1 - Z\Pi]^{-1}Z - Z$ currently does `getrf+getri+gemm`; solving $[1 - Z\Pi]W' = Z$ with `getrf+getrs` removes $\sim 40\%$ of the flops and one `gemm`. When the (ω, q) block is rank-local (the common case) call `cuSOLVER` directly, skipping the per-call SLATE object construction (7 per (ω, q) pair). Remove the diagnostic `device_sync()` serialization in production builds.

P8 — Fix the device ISDF builder ≥ 16 ranks (B3); Σ divergence correction.

(Effort M, risk M.) Trace the `Zqur`-stage device pointer that reaches a host-only MPI path; route through NCCL like the builder's other collectives, or stage that single exchange. Until then the `save=/load` workaround stands. `Sigma_div_correction` is host-only scalar $O(n_s n_k n_b^2 N_p)$ per iteration — port or thread before `kp444`-scale GPU production.

P9 — Write path. −20–30 s/iter-equiv.

(Effort S, risk L.) Rank-0 serial HDF5 of ~ 35 GB per iteration (85–90 s per `kp222` GPU run). Parallel HDF5 or an async writer thread; optionally checkpoint every N iterations.

5.1 Projected outcome (kp222/500b, 2 nodes · 8×A100)

Per SCF iteration	today	after P1–P7	change
update_w	162 s	≈ 38 s	redistributes 111→ 47 measured (P1) →12 with double-buffering, W-solve 17→11, mirror/allocs 13→1, Π 20→14
THC-GW Σ	50 s	≈ 15 s	$G_{ij} \rightarrow G_{uv}$ 18→2; host reduce partially remains
Host phases (/iter-equiv.)	~ 110 s	≈ 45 s	device Dyson, capped μ , async write
Total per iteration	≈ 320 s	≈ 100 s	vs. CPU 2n ≈ 340 s $\Rightarrow \approx 3.5\times$ wall, $\approx 5\times$ solver

At `kp444` the removed terms scale with n_q and rank count, so the node-for-node advantage grows; the W-solve roofline alone bounds it at $\approx 11\times$ (16 A100 nodes vs. 16 Genoa nodes). A realistic production target is **5–8× node-for-node**, at which point the remaining gap is irreducible communication.

5.2 Suggested sequencing

- Week 1** — done: P2 pool built and wired (opt-in; not usable at `kp222/500b`, see P2) → P1 device-direct redistribute implemented, validated bit-identical, benchmarked at 8 and 16 ranks, and made the default where the MPI supports it. Next in this line: double-buffer the exchange across two streams to close the remaining 47 s→12 s.
- Week 2**: P3 + P4 + P5 (transfer hygiene); B4 μ -walk cap. Re-run `kp222`; launch `kp444` GPU.
- Week 3+**: P6 (device Dyson + eigenspectra memory), P7, P9, then P8 (builder fix) to retire the ERI-prebuild workaround.

6 Run inventory

Under `~/ceph/CoQui/GPU_PORT_run/` on rusty:

- `si_kp222_n500_e125/bench_{cpu,gpu}_*` — `kp222/500b` matrix: CPU 1n/2n; A100 2n/4n; H100 1n/2n (post-fix logs: jobs 6674560–62, 6676687, 6678589–90); `eri_n500.h5` (2.9 GB).

- `si_kp222_n250_e125/`, `si_kp444_n500_e125/` — 250-band single-node pair; kp444 CPU 8n (6671589); `eri_n500.h5` (23 GB); pending GPU: 64×A100 (6678594), 48×H200 (6678593).
- `correctness/` — shared-ERI validation matrix, CPU 1/2/4/8 ranks + GPU H100 1/4 ranks; all $E = 0.5146018573397(1-4)$ a.u.
- `GPU_PORT/src` — branch `gpu`; binaries `build/cpu`, `build/gpu` (sm80), `build/gpu90` (sm90); fixes 80f4a5d, 765754c pushed to `origin/gpu`.